BITACORA – LAB07

Alumno: Sanchez Mayen Tristan Qesen

⇒ Parte 0 · Ver el problema antes de arreglarlo

Para empezar esta practica primero es importante ver el estado actual del sistema antes de que hagamos cambios. Para ello primero verificarremos el estado de las cuentas:

```
~/…/mexi-banco-s08 $ curl -s http://localhost:8080/api/cuentas/002180009217939901
{"id":6,"clabe":"002180009217939901","titular":"Ana","saldo":1000.00}
```

```
~/…/mexi-banco-s08 $ curl -s http://localhost:8080/api/cuentas/002180009217939902
{"id":7,"clabe":"002180009217939902","titular":"Beto","saldo":500.00}
```

Podemos ver que Ana tiene un saldo de 750 y Beto uno de 700.

Ahora levantaremos transferencia y apagaremos las cuentas para ver en que resulta. El resultado deberia de ser que se envio el dinero pero no se recibio.

```
PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
```

```
curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180009217939901","claveDestino":"002180009217939902","monto":200}'
```

```
docker compose stop cuenta
```

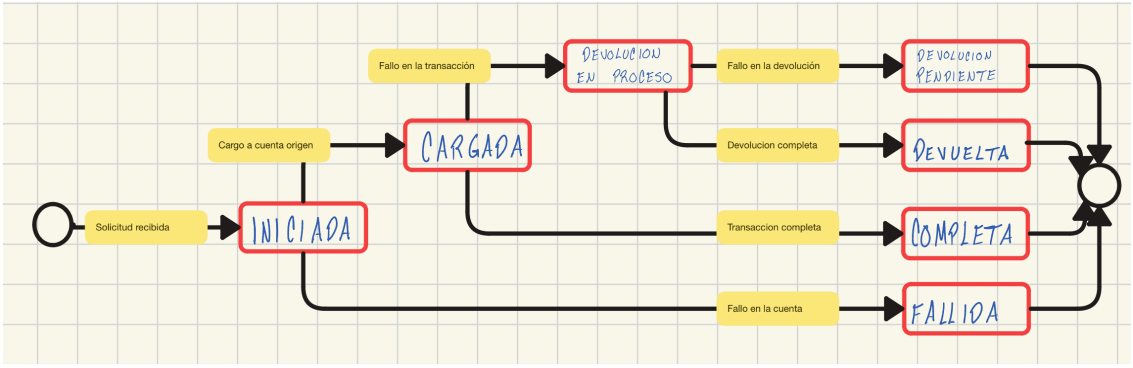
```
$ curl http://localhost:8080/api/cuentas/002180009217939901
{"id":6,"clabe":"002180009217939901","titular":"Ana","saldo":800.00}

$ curl http://localhost:8080/api/cuentas/002180009217939902
{"id":7,"clabe":"002180009217939902","titular":"Beto","saldo":500.00}
```

¿Por qué paso?

El dinero quedó en el aire porque en una arquitectura distribuida cada petición HTTP confirma su transacción local de forma independiente al no existir una transacción global ACID que abarque múltiples servicios, la caída de “cuenta” hizo que la transferencia a Beto no se realizara y ademas al no estra programado el reembolso para Ana, ella perdio sus 200 pesos ☹.

⇒ Parte 1 · La saga orquestada (obligatoria)



⇒ El codigo clave

A. Transferencia.java y EstadoTransferencia.java

Se añadió la columna estado (enum EstadoTransferencia) y detalle para registrar en qué paso se encuentra la saga en la base de datos de Postgres:

```java
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 30)
private EstadoTransferencia estado;

@Column(length = 500)
private String detalle;
```

B. TransferenciaService.java

Al recibir la solicitud, se guarda la transferencia con estado “INICIADA” para tener trazabilidad ante una caída del servicio.

```java
CuentaRemota origen = cuentas.cargar(claveOrigen, monto);
t.setEstado(EstadoTransferencia.CARGADA);
t.setDetalle("Cargo aplicado en cuenta origen. Saldo resultante: " + origen.saldo());
t = transferencias.save(t);
```

Si el abono falla, se invoca compensar()

```java
try {
    destino = cuentas.abonar(claveDestino, monto);
} catch (Exception eAbono) {
    compensar(t, claveOrigen, claveDestino, monto, eAbono.getMessage());
    return t;
}
```

Por ultimo incluimos la logica para realizar la devolucion

```java
private void compensar(Transferencia t, String claveOrigen, String claveDestino, BigDecimal monto, String
errorOriginal) {
    t.setEstado(EstadoTransferencia.COMPENSANDO);
    t.setDetalle("Abono a destino fallo (" + errorOriginal + "). Ejecutando compensacion en origen.");
    transferencias.save(t);

    CuentaRemota origenReembolsado = null;
    int maxReintentos = 3;
    long esperaMs = 500;

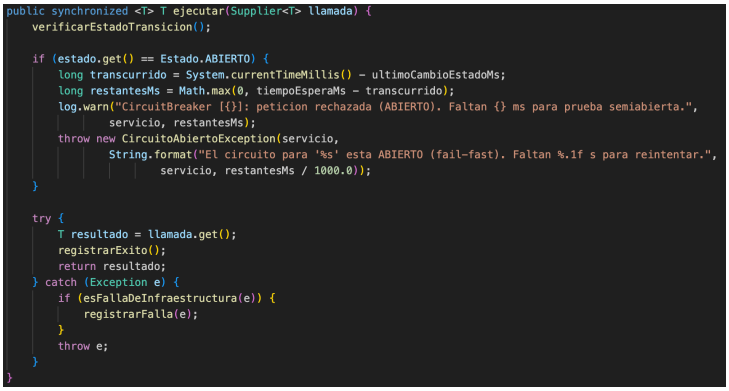
    for (int intento = 1; intento <= maxReintentos; intento++) {
        try {
            origenReembolsado = cuentas.abonar(claveOrigen, monto);
            break;
        } catch (Exception e) {
            Thread.sleep(esperaMs);
            esperaMs *= 2; // Espera creciente
        }
    }

    if (origenReembolsado != null) {
        movimientos.registrar(claveOrigen, TipoMovimiento.DEPOSITO, monto, origenReembolsado.saldo(),
            "Compensacion: transferencia revertida a " + claveDestino);
        t.setEstado(EstadoTransferencia.COMPENSADA);
        transferencias.save(t);
        throw new TransferenciaCompensadaException(t.getId(), "... Estado: COMPENSADA.");
    } else {
        t.setEstado(EstadoTransferencia.COMPENSACION_PENDIENTE);
        transferencias.save(t);
        throw new TransferenciaFallidaException(t.getId(), "... Estado: COMPENSACION_PENDIENTE.");
    }
}
```

C. CircuitBreaker.java

El CircuitBraker tiene 4 bloques de codigo que considero que son clave:

1. Metodo central: En esta seccion del codigo es donde se envuelven las las llamadas HTTP hacia 'cuenta'. Aquí tambien se encuentra la logica de cuando el circuito este abierto o cerrado.
   En caso de estar abierto hara un rechazo inmediato (Fail-Fast) sin tocar la red.
   En caso de estar cerrado solo ejecutara y evaluara el resultado.

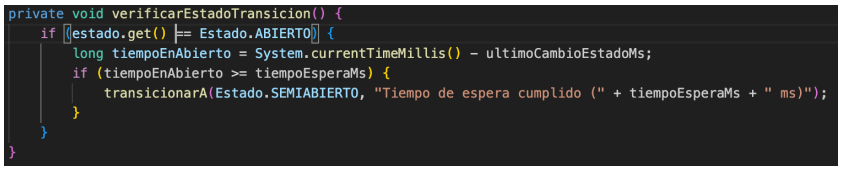


```
public synchronized <T> T ejecutar(Supplier<T> llamada) {
    verificarEstadoTransicion();

    if (estado.get() == Estado.ABIERTO) {
        long transcurrido = System.currentTimeMillis() - ultimoCambioEstadoMs;
        long restantesMs = Math.max(0, tiempoEsperaMs - transcurrido);
        log.warn("CircuitBreaker [{}]: peticion rechazada (ABIERTO). Faltan {} ms para prueba semiabierta.",
                servicio, restantesMs);
        throw new CircuitoAbiertoException(servicio,
                String.format("El circuito para '%s' esta ABIERTO (fail-fast). Faltan %.1f s para reintentar.",
                        servicio, restantesMs / 1000.0));
    }

    try {
        T resultado = llamada.get();
        registrarExito();
        return resultado;
    } catch (Exception e) {
        if (esFallaDeInfraestructura(e)) {
            registrarFalla(e);
        }
        throw e;
    }
}
```

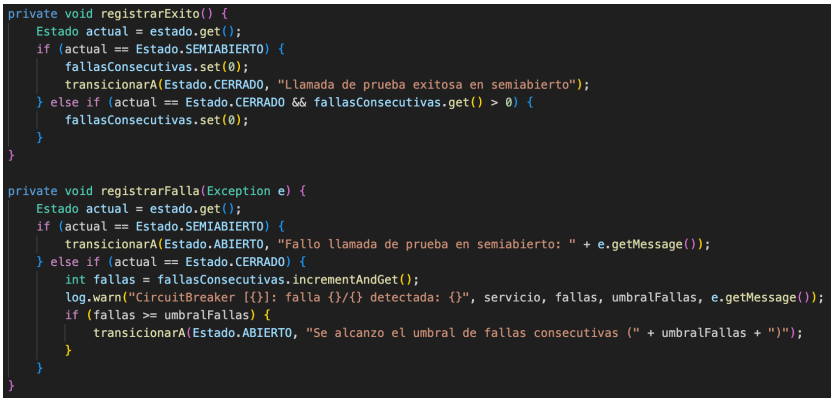
2. Transicion de abierto a semiabierto: Permite que el circuito no se quede abierto para siempre. Al cumplirse la ventana de espera, cambia automáticamente a SEMIABIERTO para permitir una petición de prueba que verifique si 'cuenta' ya revivió.



```
private void verificarEstadoTransicion() {
    if (estado.get() == Estado.ABIERTO) {
        long tiempoEnAbierto = System.currentTimeMillis() - ultimoCambioEstadoMs;
        if (tiempoEnAbierto >= tiempoEsperaMs) {
            transicionarA(Estado.SEMIABIERTO, "Tiempo de espera cumplido (" + tiempoEsperaMs + " ms)");
        }
    }
}
```

3. Transicion del estado: Aquí esta la logica de cuando el circuito se abre o se cierra:

   a. Si está en CERRADO y llega a 3 fallas consecutivas, se cambia a ABIERTO.
   b. Si está en SEMIABIERTO y la llamada de prueba falla, regresa de inmediato a ABIERTO.
   c. Si la prueba en SEMIABIERTO tiene éxito, el circuito se restablece a CERRADO y reinicia el contador a 0.



```
private void registrarExito() {
    Estado actual = estado.get();
    if (actual == Estado.SEMIABIERTO) {
        fallasConsecutivas.set(0);
        transicionarA(Estado.CERRADO, "Llamada de prueba exitosa en semiabierto");
    } else if (actual == Estado.CERRADO && fallasConsecutivas.get() > 0) {
        fallasConsecutivas.set(0);
    }
}

private void registrarFalla(Exception e) {
    Estado actual = estado.get();
    if (actual == Estado.SEMIABIERTO) {
        transicionarA(Estado.ABIERTO, "Fallo llamada de prueba en semiabierto: " + e.getMessage());
    } else if (actual == Estado.CERRADO) {
        int fallas = fallasConsecutivas.incrementAndGet();
        log.warn("CircuitBreaker [{}]: falla {}/{} detectada: {}", servicio, fallas, umbralFallas, e.getMessage());
        if (fallas >= umbralFallas) {
            transicionarA(Estado.ABIERTO, "Se alcanzo el umbral de fallas consecutivas (" + umbralFallas + ")");
        }
    }
}
```

4. Clasificar los errores: Esta seccion se tiene para que no se abra el circuito cuando sean errores como un error 404 (cuenta inexistente) o 422 (sin saldo) demuestra que el microservicio cuenta sí está vivo y respondiendo correctamente. Contar errores de negocio como fallas que puedan abrir el circuito sería un error de diseño; este filtro asegura que el circuito solo se abra ante caídas reales de infraestructura.

⇒ Compensacion funcionando

Para comprobar que funcione el sistema de realizar un reembolso en caso de que no se complete la transferencia se probo primero enviando la transferencia de 150 de Ana a Carlos. Pero antes hacemos una consulta de los saldos para corroborar que se hace la devolucion:

```
$ curl http://localhost:8080/api/cuentas/002180009217939901
{"id":6,"clabe":"002180009217939901","titular":"Ana","saldo":700.00}

$ curl http://localhost:8080/api/cuentas/002180009217939999
{"id":8,"clabe":"002180009217939999","titular":"Carlos Falla","saldo":300.00}
```

Ahora si hacemos la transferencia:

```
$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180009217939901","claveDestino":"002180009217939999","monto":150}'
```

Y vemos la respuesta HTTP recibida:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Transferencia Compensada",
  "status": 422,
  "detail": "La transferencia fallo en el abono a destino (002180009217939999). Saga compensada: el saldo fue
devuelto a la cuenta origen (002180009217939901). Estado: COMPENSADA.",
  "instance": "/transferencias",
  "transferenciaId": 4,
  "estado": "COMPENSADA"
}
```

Volvemos a consultar los saldos:

```
$ curl http://localhost:8080/api/cuentas/002180009217939901
{"id":6,"clabe":"002180009217939901","titular":"Ana","saldo":700.00}

$ curl http://localhost:8080/api/cuentas/002180009217939999
{"id":8,"clabe":"002180009217939999","titular":"Carlos Falla","saldo":300.00}
```

Tambien podemos revisar la base de datos de las transferencias para ver el estado de esta:

```
 id |    clave_origen    |   clave_destino    | monto  |   estado   |                                    detalle
----+--------------------+--------------------+--------+------------+-------------------------------------------
-------------------------------
  4 | 002180009217939901 | 002180009217939999 | 150.00 | COMPENSADA | Abono en destino fallo. Dinero devuelto a
cuenta origen. Saldo actual: 700.00
```

O el movimiento registrado en el ledger:

```
 id |    clabe_cuenta    |   tipo   | monto  | saldo_resultante |                          referencia
----+--------------------+----------+--------+------------------+--------------------------------------------
--------
 16 | 002180009217939901 | DEPOSITO | 150.00 |           700.00 | Compensacion: transferencia revertida a
002180009217939999
```

⇒ Cuando la compensacion falla

Si ‘cuenta’ se mantiene apagada durante la pausa y durante los reintentos de compensación, la transferencia no se queda en ‘CARGADA’, entonces se actualiza y persiste en la base de datos con el estado COMPENSACION_PENDIENTE. Podemos consultar esto en la tabla de transferencias

```
 id |    clave_origen    |   clave_destino    | monto |         estado         |
detalle
----+--------------------+--------------------+-------+------------------------+-----------------------------------
-------------------------------------------------------------------------------------------------------------------
 10 | 002180009217939901 | 002180009217939902 | 50.00 | COMPENSACION_PENDIENTE | Abono fallo y compensacion al origen
tampoco pudo completarse: El circuito para 'cuenta' esta ABIERTO (fail-fast). Faltan 9.0 s para reintentar.
```

Ademas tambien se envia una respuesta recibida por el cliente:

```
{
  "title": "Compensacion Pendiente",
  "status": 503,
  "detail": "La transferencia fallo y la compensacion no pudo completarse inmediatamente. Estado:
COMPENSACION_PENDIENTE (ID: 10). Requiere reintento asincrono.",
  "estado": "COMPENSACION_PENDIENTE"
}
```

Para poder resolver y terminar con esta devolucion pendiente, podemos usar un sistema que cada cierto tiempo intente devolver el dinero a la cuenta de origen, una vez que se logre completar el proceso que se actualice el estado en la base de datos

⇒ El Circuit Breaker

Para poder consultar el estado del Circuit Breaker se puede hacer con un GET /api/transferencias/circuit-breaker, aquí se pueden tener tres estados:

1. CERRADO: En este estado el sistema funciona con normalidad, el mensaje que se tendra sera el siguiente:

   ```
   {"servicio":"cuenta","estado":"CERRADO","fallasConsecutivas":0,"umbralFallas":3,"tiempoEsperaMs":10000,"segundosResta
   ntesParaSemiabierto":0.0}
   ```

2. ABIERTO: Se entra al estado ABIERTO cuando cuenta esta apagado y haya 3 fallas consecutivas. Podemos imaginar que se recibe tres veces el error 503, cuando se recibe la tercera, el circuito pasara a abrirse y el mensaje de su estado se vera de la siguiente forma:

   ```
   {"servicio":"cuenta","estado":"ABIERTO","fallasConsecutivas":3,"umbralFallas":3,"tiempoEsperaMs":10000,"segundosResta
   ntesParaSemiabierto":10.0}
   ```

   Ademas, cualquier llamada que entre sera rechazada

   ```
   $ curl -s -w "\nTiempo: %{time_total}s | HTTP %{http_code}\n" -X POST http://localhost:8080/api/transferencias ...
   {"type":"about:blank","title":"Circuito Abierto","status":503,"detail":"El circuito para 'cuenta' esta ABIERTO (fail-
   fast). Faltan 9.9 s para reintentar.","instance":"/transferencias","servicio":"cuenta"}
   Tiempo: 0.014s | HTTP 503
   ```

3. SEMIABIERTO: Cuando pase un tiempo de ‘enfriamiento’ (10seg) el circuito pasara a SEMIABIERTO. El mensaje que se dara cuando se consulte el estado del circuit breaker sera:

   ```
   {"servicio":"cuenta","estado":"SEMIABIERTO","fallasConsecutivas":3,"umbralFallas":3,"tiempoEsperaMs":10000,"segundosR
   estantesParaSemiabierto":0.0}
   ```

   Despues de estos 10 segundos se volvera a levantar cuenta y se enviara una peticion de prueba para verificar que se haya reparado el servicio. Si esta peticion pasa la prueba, el circuito vuelve a CERRADO.

   ```
   {"servicio":"cuenta","estado":"CERRADO","fallasConsecutivas":0,"umbralFallas":3,"tiempoEsperaMs":10000,"segundosResta
   ntesParaSemiabierto":0.0}
   ```

⇒ **Lo que la Saga no te da**

En una transacción tradicional ACID con un monolito, el Aislamiento hace que los cambios intermedios de una transacción no sean visibles para otros observadores hasta que la transacción hace un COMMIT. En una Saga (ACID → BASE), el aislamiento se pierde porque cada paso es una transacción que confirma sus cambios de inmediato. Si un proceso consulta el saldo de la cuenta de origen en la ventana de tiempo que transcurre entre que el cargo fue exitoso y la compensación se ejecuta, verá que su dinero fue descontado temporalmente. Podría experimentar un rechazo por saldo insuficiente en otra operación paralela, o generar algun miedo al usuario por pensar que su saldo desapareció (me paso una vez), aun cuando tiempo después se haga la devolucion. Las sagas logran consistencia eventual pero sacrifican el aislamiento estricto, como nos lo comentaba el profesor en clases pasadas.

BITACORA – LAB08 (Corrección)

Alumno: Sanchez Mayen Tristan Qesen

⇒ Corrección solicitada: Idempotencia en la compensación

Observación de la revisión (Corte 4-nov-2026)

"S08: no explicas por qué la compensación debe ser idempotente. Cómo se corrige: Agrega un párrafo: si la devolución se aplica pero la respuesta se pierde, el reintento devolvería el dinero dos veces; di cómo lo evitarías (por ejemplo, con una clave fija por transferencia)."

¿Por qué la compensación debe ser estrictamente idempotente?

En una arquitectura distribuida que implementa una Saga orquestada (como la del servicio de `transferencia` de Mexi Banco), si el abono a la cuenta de destino falla tras haberse ejecutado exitosamente el cargo en el origen, el orquestador dispara una transacción compensatoria para reembolsar el dinero a la cuenta de origen (`cuentas.abonar(claveOrigen, monto)`). Sin embargo, en redes distribuidas reales las comunicaciones son inherentemente no confiables: puede ocurrir que el microservicio `cuenta` reciba la petición de compensación, aplique el abono en su base de datos local y descuente la deuda, pero la respuesta HTTP de éxito se pierda en la red o sufra un *timeout* antes de llegar de vuelta al orquestador.

Ante esta falta de confirmación, el orquestador no puede distinguir entre una falla previa a la ejecución o una respuesta perdida posterior, por lo que su mecanismo de reintentos (o el proceso en segundo plano para transferencias en estado `COMPENSACION_PENDIENTE`) volverá a enviar la petición de abono compensatorio. **Si la compensación no es idempotente, el reintento devolvería el dinero dos veces** (o tantas veces como reintentos se efectúen), acreditando saldo duplicado al usuario de origen, provocando descuadres en el balance contable global del banco y creando dinero de la nada. Por esta razón, en sistemas distribuidos financieros rige el principio cardinal: *reintentar sin idempotencia cobra o devuelve dos veces.*

¿Cómo evitar la doble devolución? (Mecanismo con Clave de Idempotencia)

Para evitar este problema y hacer la compensación estrictamente idempotente, cada solicitud de devolución debe viajar con una **clave de idempotencia fija y determinista vinculada a la transferencia original**.

Por ejemplo, al invocar la compensación se adjunta en el encabezado HTTP:

```
Idempotency-Key: compensacion-tx-<idTransferencia>
```

o bien dentro del payload del microservicio un identificador único de operación derivado del ID de la transferencia (e.g. `UUID` o clave fija de la transacción).

En el microservicio `cuenta`, se mantiene una tabla de operaciones procesadas:

1. Al recibir la solicitud de abono, se comprueba atómicamente si la clave de idempotencia ya fue registrada en la base de datos.
2. Si no existe, se ejecuta el reembolso en el saldo, se asienta el movimiento en el ledger y se persiste la clave de idempotencia dentro de la misma transacción local ACID.
3. Si la clave de idempotencia ya existe (porque el intento previo sí se aplicó pero su respuesta se perdió), el microservicio **no aplica el abono por segunda vez**; simplemente recupera el resultado previamente registrado y retorna de inmediato `HTTP 200 OK` con el saldo resultante original.

De esta manera, sin importar cuántas veces el orquestador reintente la compensación ante caídas de red o retrasos, el estado del saldo de la cuenta de origen solo se modifica una única vez.